

Explicação Detalhada do Código PHP

Bloco de Salvamento de Produtos – INSERT e UPDATE com Validação

1. Visão Geral do Bloco

Este trecho de código PHP é responsável por receber os dados enviados pelo formulário HTML de produtos, validar essas informações e, conforme o caso, executar uma operação de **inserção (INSERT)** ou **atualização (UPDATE)** no banco de dados MySQL. Ele é o coração do CRUD de produtos do sistema de estoque.

O bloco inteiro só é executado quando há uma requisição do tipo **POST**, ou seja, quando o usuário clicou no botão 'Cadastrar' ou 'Salvar alterações' do formulário. Caso a página seja acessada normalmente (GET), este bloco é completamente ignorado.

2. Verificação do Método HTTP

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

A variável superglobal `$_SERVER['REQUEST_METHOD']` contém o método HTTP utilizado na requisição atual. Os métodos mais comuns são GET (acessar uma página) e POST (enviar dados de um formulário). O operador `===` faz comparação estrita: verifica valor e tipo ao mesmo tempo, sendo mais seguro que `==` simples.

—■ **Resultado prático:** Somente quando o formulário for submetido, as linhas internas deste bloco serão executadas.

3. Captura e Sanitização dos Dados

```
$nome      = $mysqli->real_escape_string(trim($_POST['nome'] ?? ''));  
$quantidade = (int) ($_POST['quantidade'] ?? 0);  
$valor     = number_format((float) str_replace(',', '.', $_POST['valor'] ?? '0'), 2, '.', '');  
$id_post   = (int) ($_POST['id'] ?? 0);
```

Cada linha trata um campo diferente do formulário:

Campo	Técnicas aplicadas	Objetivo
nome	<code>\$_POST['nome'] ?? ''</code> <code>real_escape_string()</code> <code>trim()</code>	Captura o valor (ou '' se vazio), remove espaços nas bordas e escapa caracteres especiais.
quantidade	<code>\$_POST['quantidade'] ?? 0</code> <code>(int)</code>	Captura o valor (ou 0 se ausente) e força conversão para inteiro, evitando letras ou decimais.
valor	<code>\$_POST['valor'] ?? '0'</code> <code>str_replace(',', '.', ...)</code> <code>(float) + number_format()</code>	Substitui vírgula por ponto (padrão brasileiro → SQL), converte para float e formata com 2 casas decimais.

id_post	<code>\$_POST['id'] ?? 0</code> (int)	Captura o id oculto do formulário (só presente na edição). Se for 0, é um novo cadastro.
---------	--	---

■ **Por que usar `real_escape_string()`?** Sem essa função, um usuário mal-intencionado poderia digitar algo como `' OR '1'='1` no campo nome e manipular a consulta SQL — técnica conhecida como **SQL Injection**. O `real_escape_string` escapa esses caracteres, tornando-os inofensivos.

4. Bloco de Validações

```
if (empty($nome)) {
    $mensagem = "O nome do produto é obrigatório.";
    $tipo_msg = "erro";
} elseif ($quantidade < 0) {
    $mensagem = "A quantidade não pode ser negativa.";
    $tipo_msg = "erro";
} elseif ($valor < 0) {
    $mensagem = "O valor não pode ser negativo.";
    $tipo_msg = "erro";
} else {
    // executa o SQL...
}
```

Antes de tocar no banco de dados, o código verifica se os dados fazem sentido. Essa etapa chama-se **validação server-side** (validação no servidor). Mesmo que o formulário HTML tenha o atributo *required*, um usuário avançado poderia burlar essa proteção, então a validação no PHP é indispensável.

4.1 — `empty($nome)`

A função **`empty()`** retorna *true* se a variável for vazia, nula ou uma string vazia. Como já aplicamos **`trim()`** na captura, se o usuário enviar apenas espaços, o nome também será considerado vazio.

4.2 — `$quantidade < 0` e `$valor < 0`

Valores negativos de quantidade ou preço são ilógicos em um sistema de estoque. Essas verificações impedem registros inconsistentes no banco. Como `$quantidade` já foi convertida para **(int)** e `$valor` para **float**, a comparação com 0 é direta e segura.

4.3 — Estrutura `if / elseif / else`

O uso de **`elseif`** garante que apenas o *primeiro erro encontrado* seja reportado. Se o nome estiver vazio, o código não verifica quantidade nem valor — evitando múltiplas mensagens simultâneas que poderiam confundir o usuário.

5. Decisão: INSERT ou UPDATE?

```
if ($id_post > 0) {
    // ATUALIZAR
    $sql = "UPDATE produtos SET nome='$nome', quantidade=$quantidade, valor=$valor WHERE id=$id_post";
} else {
    // INSERIR
    $sql = "INSERT INTO produtos (nome, quantidade, valor) VALUES ('$nome', $quantidade, $valor)";
}
```

Essa condicional é o ponto central que distingue se o formulário está sendo usado para **criar** um produto novo ou **atualizar** um existente.

5.1 — Como o sistema sabe que é uma edição?

No formulário HTML, quando o usuário clica em 'Editar', um campo oculto `<input type="hidden" name="id">` é adicionado com o id do produto. Ao submeter, esse valor chega em `$_POST['id']` e é capturado em `$id_post`. Se `$id_post` for maior que 0, é uma edição; se for 0, é um novo cadastro.

5.2 — A instrução UPDATE

O **UPDATE** modifica registros já existentes. A cláusula **WHERE id = \$id_post** é fundamental: sem ela, todos os produtos seriam atualizados ao mesmo tempo. O `$id_post` foi convertido para (int), o que impede a injeção de um valor inválido.

5.3 — A instrução INSERT

O **INSERT INTO** adiciona uma nova linha à tabela produtos. Os campos entre parênteses definem as colunas, e os VALUES definem os dados a inserir na mesma ordem. O campo **id** não é informado pois está definido como **AUTO_INCREMENT** no banco — o MySQL gera o número automaticamente.

6. Execução da Query e Mensagem de Feedback

```
if ($mysqli->query($sql)) {  
    $mensagem = "Produto atualizado/cadastrado com sucesso.";  
    $tipo_msg = "sucesso";  
} else {  
    $mensagem = "Erro ao atualizar: " . $mysqli->error;  
    $tipo_msg = "erro";  
}
```

O método **\$mysqli->query(\$sql)** envia a instrução SQL ao banco de dados. Ele retorna *true* em caso de sucesso ou *false* em caso de falha. A variável **\$mysqli->error** armazena a mensagem de erro do MySQL, útil para depuração durante o desenvolvimento.

As variáveis **\$mensagem** e **\$tipo_msg** são usadas na view HTML para exibir um bloco colorido de feedback ao usuário — verde para sucesso, vermelho para erro, conforme as classes CSS `.mensagem.sucesso` e `.mensagem.erro` do `style.css`.

7. Limpeza do Modo de Edição

```
$produto_editar = null;
```

Após qualquer operação bem-sucedida (insert ou update), a variável `$produto_editar` é definida como **null**. Isso faz com que o formulário HTML volte ao estado de 'Cadastrar novo produto', limpando os campos preenchidos e removendo o botão 'Cancelar'. Garante que, após salvar, o painel esteja pronto para um novo cadastro.

8. Fluxograma do Bloco

O diagrama abaixo representa visualmente toda a lógica descrita neste documento, do recebimento do POST até a execução do SQL e exibição do resultado:

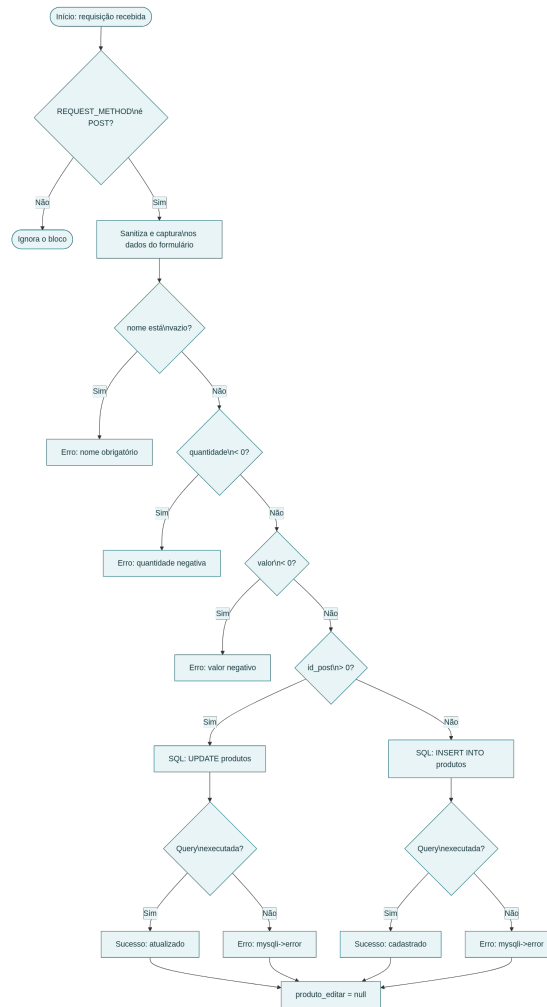


Figura 1 – Fluxo de execução do bloco de salvamento de produtos

9. Resumo das Responsabilidades

Linha / Trecho	Responsabilidade
<code>\$_SERVER['REQUEST_METHOD']</code>	Garante execução somente via POST
<code>real_escape_string() + trim()</code>	Higieniza o campo de texto
<code>(int)</code> e <code>(float)</code>	Tipagem forte dos campos numéricos
<code>str_replace + number_format</code>	Normaliza o formato do valor monetário
<code>empty()</code> / <code>< 0</code>	Validação de regras de negócio
<code>\$id_post > 0</code>	Decide entre INSERT e UPDATE
<code>\$mysqli->query()</code>	Executa a SQL no banco de dados
<code>\$mysqli->error</code>	Captura erros do MySQL

<code>\$produto_editar = null</code>	Reseta o formulário após salvar
--------------------------------------	---------------------------------

Documento gerado como material de apoio didático para o estudo de PHP e MySQL.